

Exercices sur les dictionnaires

Exercice 1 : parcours et filtrage

Soit le dictionnaire de stocks suivant :

```
1 stocks = {"pommes": 5, "bananes": 0, "cerises": 12, "kiwis": 0}
```

python

Afficher uniquement les produits dont le stock est supérieur à 0.

Exercice 2 : comptage de fréquences

À partir d'une chaîne de caractères, construire un dictionnaire qui compte le nombre d'apparitions de chaque mot.

```
1 texte = "le chat mange le chat dort le chat"
```

python

```
2 # Résultat attendu :
```

```
3 # {"le": 3, "chat": 3, "mange": 1, "dort": 1}
```

Exercice 3 : inversion de dictionnaire

Écrire une fonction `inverser(d)` qui inverse les clés et les valeurs d'un dictionnaire.

```
1 inverser({"a": 1, "b": 2, "c": 3})
```

python

```
2 # → {1: "a", 2: "b", 3: "c"}
```

Remarque : cette inversion concerne uniquement ce dictionnaire

Exercice 4 : dictionnaires imbriqués

On dispose du dictionnaire de notes suivant :

```
1 eleves = {
```

python

```
2     "Alice": {"maths": 16, "français": 14, "anglais": 18},
```

```
3     "Bob":   {"maths": 12, "français": 15, "anglais": 10},
```

```
4 }
```

Calculer et afficher la moyenne de chaque élève.

Exercice 5 : dictionnaire par compréhension imbriqué

À partir d'une liste de mots, construire un dictionnaire où chaque mot est associé à un sous-dictionnaire comptant ses voyelles et consonnes.

```
1 mots = ["python", "dictionnaire", "algorithme"]
```

python

```
2 # Résultat attendu :
```

```
3 # {"python": {"voyelles": 2, "consonnes": 4}, ...}
```

Exercice 6 : fusion intelligente

Écris une fonction `fusionner(d1, d2)` qui fusionne deux dictionnaires. Si une clé existe dans les deux, les valeurs (numériques) sont **additionnées**.

```
1 d1 = {"a": 5, "b": 3, "c": 8}
2 d2 = {"b": 2, "c": 1, "d": 4}
3 # → {"a": 5, "b": 5, "c": 9, "d": 4}
```

python

Exercice 7 : groupement par critère

Écris une fonction qui groupe une liste de mots par leur **première lettre**.

```
1 mots = ["ananas", "banane", "abricot", "cerise", "avocat", "brugnon"]
2 # → {
3 #     "a": ["ananas", "abricot", "avocat"],
4 #     "b": ["banane", "brugnon"],
5 #     "c": ["cerise"]
6 # }
```

python

Exercice 8 : top N des fréquences

Sans utiliser `collections.Counter`, écris une fonction `top_n(texte, n)` qui retourne les `n` mots les plus fréquents d'un texte sous forme de liste de tuples triés.

```
1 top_n("le chat mange le chat dort le chien mange", 2)
2 # → [("le", 3), ("chat", 2)]
```

python